

Kosmocrates HYPHAE SystemCube Extension

v0.4.3 – Symbiotic Topological Accretion and Blueprint Condensation

Companion extension to HYPHAE v0.4.1, LPCM v0.4.2, and Kosmocrates Workbench

Author: Sebastian Klemm
30. May 2026

Document status. This document is an implementation-oriented extension. It crystallizes the post-v0.4.2 design discussion: HYPHAE is not only a corpus assimilation engine for code repositories, but a symbiotic topological accretion and blueprint condensation machine. It specifies exportable SystemCube artifacts, D-density objectives, cross-language/materialization compatibility, domain adapters, and a production pipeline for turning system classes into reusable topological blueprints. It does not replace HYPHAE v0.4.1 or the LPCM v0.4.2 extension; it adds the higher-order blueprint layer above them.

Contents

Executive Summary	iv
I Doctrine	1
1 Purpose and Position	2
1.1 Purpose	2
1.2 Stack position	2
1.3 Non-goals	2
1.4 Canonical statement	2
2 Core Concepts	3
2.1 Symbiotic topological accretion	3
2.2 SystemCube	3
2.3 D-density	3
2.4 Contradiction energy	4
2.5 Host-compatible fill	4
3 Tripolar Absolute Frame	5
3.1 Meaning, coherence, phase	5
3.2 Model-internal absoluteness	5
3.3 D-energy for blueprints	5
II SystemCube Model	6
4 Package Format	7
4.1 Logical structure	7
4.2 Canonical hashing	7
5 Topology and Mesh Layers	9
5.1 Common topological frame	9
5.2 Mesh layer	9
5.3 Layer alignment	9
6 Blueprint Units	11
6.1 Blueprint unit taxonomy	11
III Production Process	12
7 System-to-Cube Capture	13
7.1 Capture pipeline	13
8 Multi-System Accretion	14
8.1 Accretion objective	14
8.2 Accretion process	14

8.3	Complement as hypothesis	14
9	System Enrichment and Injection	15
9.1	Injection is materialization-bound	15
9.2	Controlled acceleration	15
IV	Compatibility and Materialization	16
10	Compatibility Profiles	17
10.1	Language compatibility	17
10.2	Cross-language materialization	17
11	Domain Adapters	18
11.1	General adapter contract	18
11.2	Domain readiness	18
12	Expert Interaction Model	19
12.1	Expert as fitness provider	19
12.2	Human review surfaces	19
V	Energy Kernel	20
13	D-Energy Kernel	21
13.1	Existing base	21
13.2	Blueprint kernel	21
13.3	Strict D	21
14	Maximum Coherent Fill	22
14.1	Target	22
14.2	Fill decision	22
VI	API and CLI	23
15	API Extensions	24
15.1	REST endpoints	24
15.2	Build request	24
15.3	Accretion request	24
16	CLI Extensions	25
VII	Metrics and Validation	26
17	Metrics	27
17.1	Cube metrics	27
17.2	Operational metrics	27
18	Acceptance Tests	28
18.1	Unit tests	28
18.2	Integration tests	28
19	Definition of Done	29
VIII	Implementation Guidance	30
20	Recommended Implementation Order	31
20.1	Phase C0: Passive cube export	31

20.2 Phase C1: D-density report	31
20.3 Phase C2: Multi-cube alignment	31
20.4 Phase C3: Target blueprint cube	31
20.5 Phase C4: LPCM integration	31
20.6 Phase C5: Workbench materialization	31
20.7 Phase C6: Cross-language materialization	31
21 Coding Agent Notes	32
21.1 Reuse the existing energy law	32
21.2 Keep the first MVP small	32
21.3 Boundary rules	32
Closing Thesis	33

Executive Summary

HYPHAE v0.4.1 gives Kosmocrates a microtopological diagnosis and surgery planning layer. The LPCM v0.4.2 extension gives it controlled fragmentation, local support-majority condensation, seam-compatible percolation, and monotone contraction of spurious degrees of freedom. This v0.4.3 extension adds the next abstraction:

System topology becomes an exportable, reusable, materializable artifact.

The central artifact is the **SystemCube**: a canonical, content-addressed, evidence-bound topological blueprint for a system class, a host system, or a target architecture. A SystemCube is not source code. It is a structured package containing topology, mesh layers, roles, relations, constraints, test fibers, anti-patterns, NormGenes, StructuralCrystals, CollapsePlans, materialization profiles, compatibility rules, evidence links, and validation templates.

The governing thesis is:

Do not copy systems. Accrete their topology. Compress their invariants. Inject only what the host can survive.

This document defines HYPHAE as a **symbiotic topological accretor**. It may use parasitic-looking mechanisms at the surface - attaching to external systems, extracting their structural mesh, and absorbing their stable patterns - but its intended function is symbiotic: it transforms extracted structure into host-compatible, evidence-bound, gate-passed improvement mass. The result is a machine that can enrich systems without requiring the operator to manually understand every implementation detail of the domain, provided that the domain can be translated into observations, relations, constraints, and fitness checks.

Core production chain

```
System examples / host repo / expert spec / synthetic artifacts
-> observations
-> CodeHDAG or DomainHDAG
-> SystemCube projection
-> mesh layer alignment
-> D-density scoring
-> contradiction-energy filtering
-> LPCM condensation
-> SystemCube blueprint export
-> Workbench/Foundry materialization
-> parse-back / validation / memory update
```

Tripolar energy basis

The blueprint layer reuses the Kosmocrates epistemic energy function:

$$D = \psi \cdot \rho \cdot \omega \quad (1)$$

where ψ is meaning or target fit, ρ is coherence or structural stability, and ω is phase, time, or HDAG/topological coherence. A good SystemCube is not the largest cube. It is the cube with maximal target-compatible D-density under minimal contradiction energy.

What this extension adds

- SystemCube/KosmoCube as exportable topological blueprint package;
- common topological frame for aligning many system meshes;
- D-density and contradiction-energy objectives;
- mesh-layer accretion and maximal coherent host-compatible fill;
- language-specific, language-family, and language-independent compatibility profiles;
- domain-general adapter contract beyond source code;
- Cube materialization pipeline: blueprint to code/process/specification;
- APIs, CLI, metrics, acceptance tests, and Definition of Done.

Part I

Doctrine

1 Purpose and Position

1.1 Purpose

This document defines a topological blueprint layer for Kosmocrates/HYPHAE. It converts the accumulated mechanisms of Kosmocrates, HYPHAE, Metatron, LPCM, Workbench, and Foundry into a reusable production concept: systems can be captured, compared, condensed, exported, and re-materialized as topological artifacts.

1.2 Stack position

Kosmocrates/PSE
memory, ledger, D-energy, replay, governance, HDAG, QTIC
Workbench/Foundry
task execution, artifact generation, build/test/parse-back validation
HYPHAE v0.4.1
SourceCubes, CubeSwarm, CorpusCartography, Metatron microtopology
LPCM v0.4.2
fragmentation, support mass, local condensation, seam percolation
SystemCube v0.4.3
exportable topological blueprint and materialization substrate

1.3 Non-goals

This extension is not:

- a replacement for code, tests, domain experts, or Foundry validation;
- a claim that a generated SystemCube is absolutely perfect;
- a universal translator without adapters;
- a license-bypass mechanism;
- a raw code ingestion format;
- an autonomous authority surface;
- a model-training dataset format;
- a reason to bypass Kosmocrates fail-closed governance.

1.4 Canonical statement

A SystemCube is a materializable topological blueprint: it stores what a system structurally is, why that structure is supported, how it may be materialized, and how materialization must be validated.

2 Core Concepts

2.1 Symbiotic topological accretion

HYPHAE behaves like a compressor and a pump simultaneously. It extracts structural mesh from compatible systems, compresses repeated and complementary invariants, and injects only host-compatible improvement structure into the target through Workbench and Foundry.

The parasitic-looking part is extraction:

```
external systems -> observations -> source cubes -> structural yield
```

The symbiotic function is transformation:

```
structural yield -> gates -> condensation -> host-compatible blueprint
```

Invariant ACC-001: No harmful extraction semantics. The implementation **MUST** treat accretion as non-copying structural extraction. Raw external code or protected content **MUST NOT** be imported into a SystemCube by default.

2.2 SystemCube

A SystemCube is an exportable topological blueprint package. It is to a system what a CAD model is to a physical object: not the object itself, but a structured, materializable representation of its form, constraints, and validation requirements.

```
*.kcube or *.kosmocube  
= canonical system topology package
```

A SystemCube may represent:

- one existing host system;
- a class of systems;
- a target architecture derived from many systems;
- a domain workflow;
- a language-independent implementation pattern;
- a language-specific materialization target.

2.3 D-density

The best SystemCube is not the largest cube. It is the cube with maximal target-compatible D-density under minimal contradiction energy.

$$D = \psi \cdot \rho \cdot \omega \tag{2.1}$$

For a cube C , define D-density over its cells or mesh units:

$$\mathcal{D}(C) = \frac{1}{|U_C|} \sum_{u \in U_C} D(u) \cdot A(u) \quad (2.2)$$

where U_C is the set of blueprint units and $A(u)$ is admissibility mass from gates, provenance, license, taint, validation, and authority.

2.4 Contradiction energy

Contradiction energy measures incompatible structure inside the candidate cube:

$$\mathcal{K}(C) = \sum_{(u,v) \in E_C} \text{conflict}(u,v) \cdot \text{weight}(u,v) \quad (2.3)$$

The blueprint objective is:

$$\max_C \mathcal{D}(C) - \lambda \mathcal{K}(C) - \mu \mathcal{C}(C) \quad (2.4)$$

where $\mathcal{C}(C)$ is avoidable complexity. Large cubes are penalized when mass does not increase coherent target utility.

2.5 Host-compatible fill

A cube is enriched by filling voids. Voids are not arbitrary missing features. They are target-relevant absence regions in the host topology.

HostVoid = missing structure whose addition increases D-density
without exceeding contradiction or complexity budgets.

Requirement FILL-001: No mass-for-mass-sake. SystemCube enrichment **MUST NOT** maximize feature count, file count, node count, or mesh density alone. It **MUST** optimize target-compatible D-density under conflict and validation constraints.

3 Tripolar Absolute Frame

3.1 Meaning, coherence, phase

The SystemCube layer adopts the tripolar epistemic basis:

Axis	Name	Meaning
ψ	Meaning / semantic charge	Does the unit point toward the target? Does it carry relevant meaning?
ρ	Coherence / stability	Does the unit hold structurally? Does it pass gates, evidence, and stability checks?
ω	Phase / time / topology	Does the unit sit correctly in development time, HDAG phase, causal topology, and materialization order?

3.2 Model-internal absoluteness

Within the Kosmocrites model, ψ , ρ , and ω are treated as irreducible epistemic primary axes. Other quantities may be necessary, but they act as gates, constraints, modulators, or materialization conditions.

license gate	-> may zero admissibility
Foundry result	-> updates rho
host-fit	-> specializes psi
HDAG/QTIC	-> specializes omega
conflict energy	-> subtracts from objective

Invariant TRI-001: Multiplicative collapse. A candidate with zero meaning, zero coherence, or zero phase/topological fit **MUST** have zero effective D-energy unless explicitly marked as exploratory evidence.

3.3 D-energy for blueprints

$$D_{\text{blueprint}}(u, H, g) = \psi_{\text{target}}(u, H, g) \cdot \rho_{\text{structure}}(u) \cdot \omega_{\text{phase}}(u, H) \quad (3.1)$$

where H is the host and g is the goal. This yields the guiding principle:

A blueprint is strong when its units mean the right thing, hold together structurally, and arrive in the right phase of the host's evolution.

Part II

SystemCube Model

4 Package Format

4.1 Logical structure

A SystemCube package is a directory or archive with a canonical manifest.

```
example.kcube/  
  manifest.json  
  topology/  
    system_hdag.json  
    mesh_layers.json  
    regions.json  
    seams.json  
  energy/  
    d_density_report.json  
    contradiction_report.json  
    admissibility_report.json  
  memory/  
    structural_crystals.json  
    norm_genes.json  
    anti_patterns.json  
    microtopology_index.json  
  plans/  
    collapse_plans.json  
    surgery_options.json  
    materialization_targets.json  
  validation/  
    foundry_templates.json  
    parseback_expectations.json  
    acceptance_tests.json  
  evidence/  
    evidence_bundles.json  
    provenance_map.json  
    source_policy.json  
  compatibility/  
    language_profiles.json  
    domain_profiles.json  
    adapter_requirements.json  
  signatures/  
    content_hashes.json  
    qtic_certificates.json  
    replay_descriptor.json
```

4.2 Canonical hashing

Every package **MUST** have a canonical manifest digest. Each component **SHOULD** be separately content-addressed so that partial reuse is possible.

Type SystemCubeManifest.

```
pub struct SystemCubeManifest {  
  pub cube_id: Digest,
```

```
    pub format_version: String,
    pub name: String,
    pub cube_kind: CubeKind,
    pub target_domain: DomainId,
    pub topology_digest: Digest,
    pub mesh_digest: Digest,
    pub energy_digest: Digest,
    pub memory_digest: Digest,
    pub validation_digest: Digest,
    pub evidence_digest: Digest,
    pub compatibility_digest: Digest,
    pub replay_descriptor: ReplayDescriptor,
  }

pub enum CubeKind {
  HostSnapshot,
  TargetBlueprint,
  SystemClassBlueprint,
  DomainWorkflowBlueprint,
  LanguageSpecificBlueprint,
  LanguageIndependentBlueprint,
  SyntheticCandidate,
  EvidenceOnly,
}
```

Requirement PKG-001: No opaque blueprint. A SystemCube **MUST NOT** be a single opaque text blob. It **MUST** expose typed topology, evidence, energy, compatibility, and validation sections.

Requirement PKG-002: Replay descriptor. A SystemCube **MUST** carry a replay descriptor sufficient to reconstruct its canonical digest and major derived reports from the same inputs.

5 Topology and Mesh Layers

5.1 Common topological frame

A SystemCube aligns systems into a CommonTopologicalFrame. This frame is the coordinate system into which multiple systems, source cubes, synthetic artifacts, and expert specifications are projected.

Type CommonTopologicalFrame.

```
pub struct CommonTopologicalFrame {  
    pub id: Digest,  
    pub domain: DomainId,  
    pub axes: Vec<TopologyAxis>,  
    pub roles: Vec<RoleKind>,  
    pub edge_kinds: Vec<EdgeKind>,  
    pub layer_kinds: Vec<LayerKind>,  
    pub fiber_kinds: Vec<FiberKind>,  
    pub constraint_kinds: Vec<ConstraintKind>,  
    pub projection_policy: ProjectionPolicy,  
}
```

5.2 Mesh layer

Each source contributes a mesh layer. Mesh layers are aligned, compared, and condensed.

Type MeshLayer.

```
pub struct MeshLayer {  
    pub id: Digest,  
    pub source_ref: SourceRef,  
    pub projected_frame_id: Digest,  
    pub nodes: Vec<MeshNode>,  
    pub edges: Vec<MeshEdge>,  
    pub density_report: MeshDensityReport,  
    pub void_report: VoidReport,  
    pub evidence: EvidenceBundleId,  
    pub taint: TaintLabel,  
    pub license: LicenseStatus,  
}
```

5.3 Layer alignment

Layer alignment maps many meshes into the same frame.

```
Mesh_1 -> projection -> Frame F  
Mesh_2 -> projection -> Frame F  
...  
Mesh_n -> projection -> Frame F  
  
AlignedMesh = align(Mesh_1, ..., Mesh_n)
```

Gate MESH-001: Projection loss tracking. Every projection into a `CommonTopologicalFrame` **MUST** record semantic loss, structural loss, and unsupported assumptions.

Gate MESH-002: Alignment is evidence, not truth. Aligned mesh overlap **MAY** support a candidate invariant. It **MUST NOT** by itself become trusted blueprint structure.

6 Blueprint Units

6.1 Blueprint unit taxonomy

```
pub enum BlueprintUnitKind {
    Role,
    Interface,
    Boundary,
    Fiber,
    Constraint,
    StateTransition,
    TestFiber,
    ValidationRule,
    MaterializationHint,
    AntiPatternMarker,
    NormGeneRef,
    StructuralCrystalRef,
}
```

Type BlueprintUnit.

```
pub struct BlueprintUnit {
    pub id: Digest,
    pub kind: BlueprintUnitKind,
    pub frame_location: FrameLocation,
    pub semantic_label: String,
    pub required_relations: Vec<BlueprintRelationId>,
    pub constraints: Vec<ConstraintRef>,
    pub d_energy: DEnergy,
    pub admissibility: AdmissibilityVector,
    pub evidence: Vec<EvidenceRef>,
    pub materialization_profiles: Vec<MaterializationProfileId>,
}
```

Type BlueprintRelation.

```
pub struct BlueprintRelation {
    pub id: Digest,
    pub from: BlueprintUnitId,
    pub to: BlueprintUnitId,
    pub relation_kind: RelationKind,
    pub phase_order: Option<PhaseOrder>,
    pub seam_requirements: Vec<SeamRequirement>,
    pub d_energy: DEnergy,
    pub contradiction_risk: Q16,
}
```

Requirement UNIT-001: Units must be materialization-aware. A BlueprintUnit **SHOULD** carry at least one materialization profile or be explicitly marked as abstract-only.

Requirement UNIT-002: Anti-pattern preservation. Rejected or harmful structures **SHOULD** be retained as AntiPatternMarkers to prevent reintroduction.

Part III

Production Process

7 System-to-Cube Capture

7.1 Capture pipeline

Input System

- > adapter-specific observation stream
- > DomainHDAG or CodeHDAG
- > topology regions
- > mesh layers
- > D-energy assignment
- > SystemCube snapshot

Algorithm BuildSystemCube.

```
function build_system_cube(input, domain_adapter, policy):
    observations = domain_adapter.observe(input)
    hdag = domain_adapter.lower_to_hdag(observations)
    regions = partition_regions(hdag, policy.region_policy)
    micrographs = metatron_project(regions)
    mesh = build_mesh_layers(hdag, micrographs)
    energy = compute_d_density(mesh, policy.energy_kernel)
    evidence = build_evidence_bundle(input, observations, hdag, mesh)
    return canonical_system_cube(mesh, energy, evidence, compatibility)
```

Requirement CAP-001: Adapter contract. A domain adapter **MUST** define entities, relations, constraints, evidence, and validation targets. If it cannot define these, the domain is not ready for SystemCube capture.

8 Multi-System Accretion

8.1 Accretion objective

Given system cubes $C_1, \dots, C_n$ and a host H , HYPHAE builds a target cube C^* :

$$C^* = \arg \max_C (\mathcal{D}(C, H, g) - \lambda \mathcal{K}(C) - \mu \mathcal{C}(C)) \quad (8.1)$$

subject to gates, evidence, license, host compatibility, and validation requirements.

8.2 Accretion process

```
Source SystemCubes
-> normalize compatibility profiles
-> project into CommonTopologicalFrame
-> compute mesh overlap and complement
-> identify host voids
-> generate candidate blueprint units
-> score D-density
-> apply contradiction-energy filters
-> LPCM local condensation
-> seam percolation
-> export TargetBlueprint cube
```

Type AccretionRun.

```
pub struct AccretionRun {
    pub id: Digest,
    pub host_cube_id: Digest,
    pub goal_id: Digest,
    pub source_cube_ids: Vec<Digest>,
    pub common_frame_id: Digest,
    pub aligned_mesh_digest: Digest,
    pub lcm_run_id: Option<Digest>,
    pub result_cube_id: Option<Digest>,
    pub report: AccretionReport,
}
```

Requirement ACC-002: Host-relative optimization. Accretion **MUST** be host-relative and goal-relative. A globally large or dense cube **MUST NOT** outrank a smaller cube with higher host-compatible D-density.

8.3 Complement as hypothesis

Complement structure can propose missing host regions, but it cannot authorize materialization.

Gate COMP-001: Complement is hypothesis. A complement-derived unit **MUST** be marked as hypothesized until evidence, seams, D-energy, and validation pass.

9 System Enrichment and Injection

9.1 Injection is materialization-bound

The blueprint layer may propose host enrichment. It cannot directly inject changes into a host.

```
SystemCube TargetBlueprint
-> HostTargetCollapsePlan
-> Workbench task
-> generated artifact or patch
-> Foundry validation
-> parse-back topology
-> Kosmocrates memory update
```

Gate INJECT-001: No direct mutation. SystemCube materialization **MUST** occur only through Workbench and Foundry. SystemCube generation **MUST NOT** mutate host code, processes, or configurations directly.

Gate INJECT-002: Host survival. A proposed injection **MUST** include rollback, validation, and parse-back expectations. If the host cannot survive the change under validation policy, the injection is rejected or held.

9.2 Controlled acceleration

The system may accelerate structural maturation, but it does not remove material constraints.

The machine does not turn any seed into any tree. It enriches a compatible host with compatible structure under real validation.

Part IV

Compatibility and Materialization

10 Compatibility Profiles

10.1 Language compatibility

A cube may be language-specific, language-family, or language-independent.

Profile	Meaning
Language-specific	Requires features of one language/runtime, e.g. Rust ownership or TypeScript types.
Language-family	Applies to related implementation families, e.g. service-oriented async web APIs.
Language-independent	Specifies roles, boundaries, states, and validation independent of concrete syntax.

Type `CubeCompatibilityProfile`.

```
pub struct CubeCompatibilityProfile {  
    pub id: Digest,  
    pub domain: DomainId,  
    pub supported_languages: BTreeMap<LanguageId, CompatibilityScore>,  
    pub supported_runtimes: Vec<RuntimeRequirement>,  
    pub required_capabilities: Vec<CapabilityRequirement>,  
    pub forbidden_targets: Vec<TargetConstraint>,  
    pub adapter_requirements: Vec<AdapterRequirement>,  
    pub materialization_profiles: Vec<MaterializationProfileId>,  
}
```

Requirement COMPAT-001: Compatibility explicit. A `SystemCube` **MUST** declare its compatibility profile. If compatibility is unknown, it **MUST** be marked as `UnknownCompatibility` and excluded from automatic materialization.

10.2 Cross-language materialization

Cross-language materialization requires a language adapter.

```
LanguageIndependentCube  
-> RustMaterializer  
-> Rust project patch  
-> cargo check/test  
-> parse-back CodeHDAG  
-> topology equivalence report
```

Gate LANG-001: Parse-back required. A cube materialized into a programming language **SHOULD** be parsed back into a `CodeHDAG` and compared against the source cube's required topology.

11 Domain Adapters

11.1 General adapter contract

To apply SystemCubes beyond code, a domain must provide a DomainAdapter.

Type DomainAdapter.

```
pub trait DomainAdapter {  
    fn domain_id(&self) -> DomainId;  
    fn observe(&self, input: DomainInput) -> Result<Vec<Observation>, AdapterError>;  
    fn lower_to_hdag(&self, observations: &[Observation]) -> Result<DomainHDAG, AdapterError>;  
    fn partition_regions(&self, hdag: &DomainHDAG) -> Result<Vec<TopologyRegion>, AdapterError>;  
    fn materialization_targets(&self) -> Vec<MaterializationTarget>;  
    fn validation_templates(&self) -> Vec<ValidationTemplate>;  
}
```

11.2 Domain readiness

A domain is ready for SystemCube treatment when it can express:

- entities;
- relations;
- roles;
- constraints;
- states;
- flows;
- failure modes;
- evidence;
- validation conditions.

Requirement DOM-001: No adapter, no universality. HYPHAE MUST NOT claim operational support for a domain without a DomainAdapter and validation templates.

12 Expert Interaction Model

12.1 Expert as fitness provider

A domain expert does not need to implement all details. The expert must provide goal, constraints, examples, counterexamples, and fitness criteria.

```
Expert provides:
  target definition
  valid examples
  invalid examples
  non-negotiable constraints
  acceptance tests
  domain-specific risk boundaries
```

```
System provides:
  topology capture
  candidate generation
  D-density scoring
  contradiction filtering
  collapse planning
  materialization suggestions
```

12.2 Human review surfaces

- goal definition;
- compatibility profile approval;
- high-risk injection approval;
- unknown domain assumption review;
- materialized artifact acceptance;
- normative promotion to trusted pattern.

Gate EXPERT-001: Expert signoff for foreign domains. For non-code domains or safety-relevant domains, SystemCube materialization **SHOULD** require domain-expert signoff before operational use.

Part V

Energy Kernel

13 D-Energy Kernel

13.1 Existing base

The SystemCube layer should reuse the Kosmocrates tripolar energy law as its base kernel:

$$D = \psi \cdot \rho \cdot \omega \quad (13.1)$$

For retrieval this may be a soft modulation. For assimilation and blueprint materialization it must become stricter.

13.2 Blueprint kernel

Type DEnergy.

```
pub struct DEnergy {
    pub psi_target_fit: Q16,
    pub rho_structural_coherence: Q16,
    pub omega_phase_coherence: Q16,
    pub admissibility: Q16,
    pub conflict_penalty: Q16,
    pub effective_d: Q16,
}
```

Type EnergyKernel.

```
pub trait EnergyKernel {
    fn score_unit(&self, unit: &BlueprintUnit, ctx: &EnergyContext) -> DEnergy;
    fn score_relation(&self, rel: &BlueprintRelation, ctx: &EnergyContext) -> DEnergy;
    fn score_cube(&self, cube: &SystemCube, ctx: &EnergyContext) -> CubeEnergyReport;
}
```

13.3 Strict D

$$D_{strict} = \psi \cdot \rho \cdot \omega \cdot A \cdot (1 - K) \quad (13.2)$$

where A is admissibility and K is normalized conflict penalty.

```
A = license * taint * governance * evidence * foundry * seam * authority
```

Hard gates may set $A = 0$.

Gate ENERGY-001: Hard-gate zeroing. A hard negative gate **MUST** zero effective D-energy for the affected unit or relation.

Gate ENERGY-002: Retrieval energy is not assimilation energy. The retrieval-optimized D implementation **MAY** use soft modulation. Assimilation and materialization scoring **SHOULD** use stricter admissibility and conflict terms.

14 Maximum Coherent Fill

14.1 Target

The target cube is the maximal coherent host-compatible fill. It is not maximal volume.

$$C^* = \text{MaxCoherentFill}(H, g, S) \quad (14.1)$$

where H is the host, g is the goal, and S is the source space.

14.2 Fill decision

A candidate unit may enter the cube only if:

1. it increases D-density;
2. it does not exceed conflict budget;
3. it has admissible evidence;
4. it passes compatibility profile;
5. it has a validation path;
6. it is seam-compatible with existing units or explicitly isolated.

Gate FILL-002: Coherence over completeness. If completeness and coherence conflict, coherence **MUST** win unless the operator explicitly chooses an exploratory or diagnostic cube.

Part VI

API and CLI

15 API Extensions

15.1 REST endpoints

```
POST /api/systemcube/build
POST /api/systemcube/accrete
GET /api/systemcube/{id}
GET /api/systemcube/{id}/manifest
GET /api/systemcube/{id}/energy
GET /api/systemcube/{id}/compatibility
GET /api/systemcube/{id}/collapse-plans
POST /api/systemcube/{id}/materialize
POST /api/systemcube/{id}/export
POST /api/systemcube/import
```

15.2 Build request

```
pub struct SystemCubeBuildRequest {
    pub input_ref: InputRef,
    pub domain_adapter: DomainAdapterId,
    pub cube_kind: CubeKind,
    pub goal: Option<String>,
    pub compatibility_target: Option<CompatibilityTarget>,
    pub policy_profile: PolicyProfileId,
    pub dry_run: bool,
}
```

15.3 Accretion request

```
pub struct SystemCubeAccretionRequest {
    pub host_cube_id: Digest,
    pub source_cube_ids: Vec<Digest>,
    pub goal_id: Digest,
    pub energy_policy: EnergyPolicy,
    pub lpcm_policy: LpcmPolicy,
    pub output_kind: CubeKind,
    pub dry_run: bool,
}
```

16 CLI Extensions

```
kosmocrates cube build --input . --adapter code --kind host-snapshot
kosmocrates cube accrete --host host.kcube --sources corpus/*.kcube --goal "harden API tests"
kosmocrates cube energy target.kcube
kosmocrates cube compatibility target.kcube --language rust
kosmocrates cube materialize target.kcube --target rust --out ./generated
kosmocrates cube parseback --cube target.kcube --artifact ./generated
kosmocrates cube export target.kcube --format kcube
```

Requirement CLI-001: Dry-run first. Materialization commands **SHOULD** default to dry-run or require explicit confirmation unless a policy profile allows unattended operation.

Part VII

Metrics and Validation

17 Metrics

17.1 Cube metrics

Metric	Meaning
D-density	Mean effective D over blueprint units.
Conflict energy	Normalized contradiction and incompatibility mass.
Void fill ratio	Share of target-relevant host voids addressed.
Semantic loss	Loss during projection into common topological frame.
Compatibility coverage	Supported materialization targets.
Foundry feasibility	Fraction of units with validation templates.
Parse-back fidelity	Match between materialized artifact and source cube topology.
NormGene support	Degree of memory-backed pattern support.
Anti-pattern suppression	Rejected harmful structures retained and avoided.
Coherence-over-completeness delta	Utility gained by excluding conflicting mass.

17.2 Operational metrics

- time to build host cube;
- time to accrete target cube;
- number of source cubes admitted/rejected;
- D-density before and after accretion;
- host patch success rate;
- repeated-host speedup;
- materialization language success rate;
- expert review burden;
- false enrichment rate;
- rollback rate.

18 Acceptance Tests

18.1 Unit tests

```
AT-CUBE-001 manifest canonical digest stable
AT-CUBE-002 opaque blueprint rejected
AT-CUBE-003 projection loss recorded
AT-CUBE-004 language compatibility unknown blocks materialization
AT-CUBE-005 hard gate zeros effective D
AT-CUBE-006 larger cube loses to smaller higher-D cube
AT-CUBE-007 contradiction energy penalizes inconsistent units
AT-CUBE-008 anti-pattern marker prevents reintroduction
AT-CUBE-009 parse-back expectation is required for code targets
AT-CUBE-010 DomainAdapter missing validation templates blocks operational cube
AT-CUBE-011 accretion is host-relative
AT-CUBE-012 complement-derived units start as hypotheses
AT-CUBE-013 exported package round-trips with identical digest
AT-CUBE-014 materialization path cannot bypass Workbench
AT-CUBE-015 Foundry failure lowers future rho or memory support
```

18.2 Integration tests

```
IT-CUBE-001 build Rust API host cube:
  parse repository, create CodeHDAG, partition regions, export host.kcube.

IT-CUBE-002 accrete API test blueprint:
  align multiple source cubes, identify test-fiber void, produce target.kcube
  with higher D-density and acceptable conflict energy.

IT-CUBE-003 materialize to Rust:
  target.kcube produces a Workbench task, Foundry cargo check/test passes,
  parse-back verifies expected Route->Handler->Service->TestFiber topology.

IT-CUBE-004 cross-language blueprint:
  language-independent CRUD cube materializes to Rust and TypeScript profiles;
  both parse-back to equivalent topology classes.

IT-CUBE-005 non-code domain readiness:
  domain without entities/relations/validation templates is rejected as not cube-ready.
```

19 Definition of Done

This extension is implemented when:

1. SystemCube manifest and package structure exist.
2. Cubes can be built from at least one code repository adapter.
3. Cubes carry topology, mesh, evidence, energy, compatibility, and validation sections.
4. D-density and contradiction energy are computed.
5. Accretion can compare multiple source cubes against a host cube.
6. Max coherent fill prefers smaller higher-D structures over larger contradictory structures.
7. Compatibility profiles gate materialization.
8. Workbench/Foundry materialization path exists for at least one language target.
9. Parse-back verifies materialized topology against cube expectations.
10. Export/import round-trip preserves canonical digest.
11. Anti-pattern and negative evidence records are retained.
12. All AT-CUBE and IT-CUBE tests pass.

Part VIII

Implementation Guidance

20 Recommended Implementation Order

20.1 Phase C0: Passive cube export

Build host snapshot cubes from existing repositories. Do not accrete or materialize yet.

20.2 Phase C1: D-density report

Implement D-energy computation over blueprint units and relations. Reuse existing Kosmocrates D-energy primitives where possible.

20.3 Phase C2: Multi-cube alignment

Align two or more SystemCubes into a CommonTopologicalFrame and emit mesh overlap/contradiction reports.

20.4 Phase C3: Target blueprint cube

Create a TargetBlueprint cube by selecting candidate units that improve host-relative D-density.

20.5 Phase C4: LPCM integration

Feed candidate unit sets into LPCM as local fragments and collapse candidates.

20.6 Phase C5: Workbench materialization

Allow selected CollapsePlanSteps to become Workbench tasks and Foundry runs.

20.7 Phase C6: Cross-language materialization

Add a second target language and parse-back equivalence tests.

21 Coding Agent Notes

21.1 Reuse the existing energy law

The agent should not invent a disconnected scoring system. It should create an EnergyKernel abstraction that can wrap or reuse the existing tripolar Kosmocrates formula.

```
Base: D = psi * rho * omega
Blueprint: D_strict = psi * rho * omega * admissibility * (1 - conflict)
```

21.2 Keep the first MVP small

The first useful MVP is:

```
Input: one Rust repository
Output: host.kcube + d_density_report + compatibility profile
No materialization required.
```

Second MVP:

```
Input: host.kcube + 3 source.kcube files
Output: target.kcube with improved test-fiber blueprint
No direct code mutation.
```

Third MVP:

```
Input: target.kcube
Output: Workbench task + Foundry-validated patch + parse-back report
```

21.3 Boundary rules

- SystemCubes do not write code.
- SystemCubes do not override governance.
- SystemCubes do not make expertise unnecessary; they structure expert input.
- Larger cubes are not automatically better.
- Cross-language compatibility must be proven by parse-back, not claimed.
- Non-code domains require adapters and validation templates.

Closing Thesis

HYPHAE began as a mechanism for external corpus assimilation. With SystemCubes, it becomes a production infrastructure for topological blueprints. The system does not aim to collect maximal mass. It aims to create maximal coherent target-compatible structure under the tripolar energy law of meaning, coherence, and phase.

The perfect host is not the largest cube. It is the cube with maximal D-density, minimal contradiction energy, and a valid path to materialization.

The SystemCube layer turns software and other formalizable systems into reusable, materializable structure files. HYPHAE accretes topology. LPCM condenses it. Metatron diagnoses it. Workbench materializes it. Foundry tests it. Kosmocrates remembers what survives.

A SystemCube is compressed system form: not code, not prose, but evidence-bound topology that can become code, process, architecture, or future memory.